

DOSSIER D'ARCHITECTURE TECHNIQUE

Vues applicative, metier, infrastructure et securite

Version : 1.0

IUT PTUT - Mars 2026

Migration bidirectionnelle WatchGuard Fireware <> pfSense

Document CONFIDENTIEL - Usage interne IUT PTUT

Sommaire

1. Vue applicative

NETMORPH est une application desktop monocouche composée de 6 modules Python et d'un ensemble de ressources statiques. L'application ne nécessite aucun serveur, aucune base de données et fonctionne entièrement en mode offline (sauf option IA). Toutes les données sont stockées localement sous forme de fichiers XML dans le répertoire de l'application.

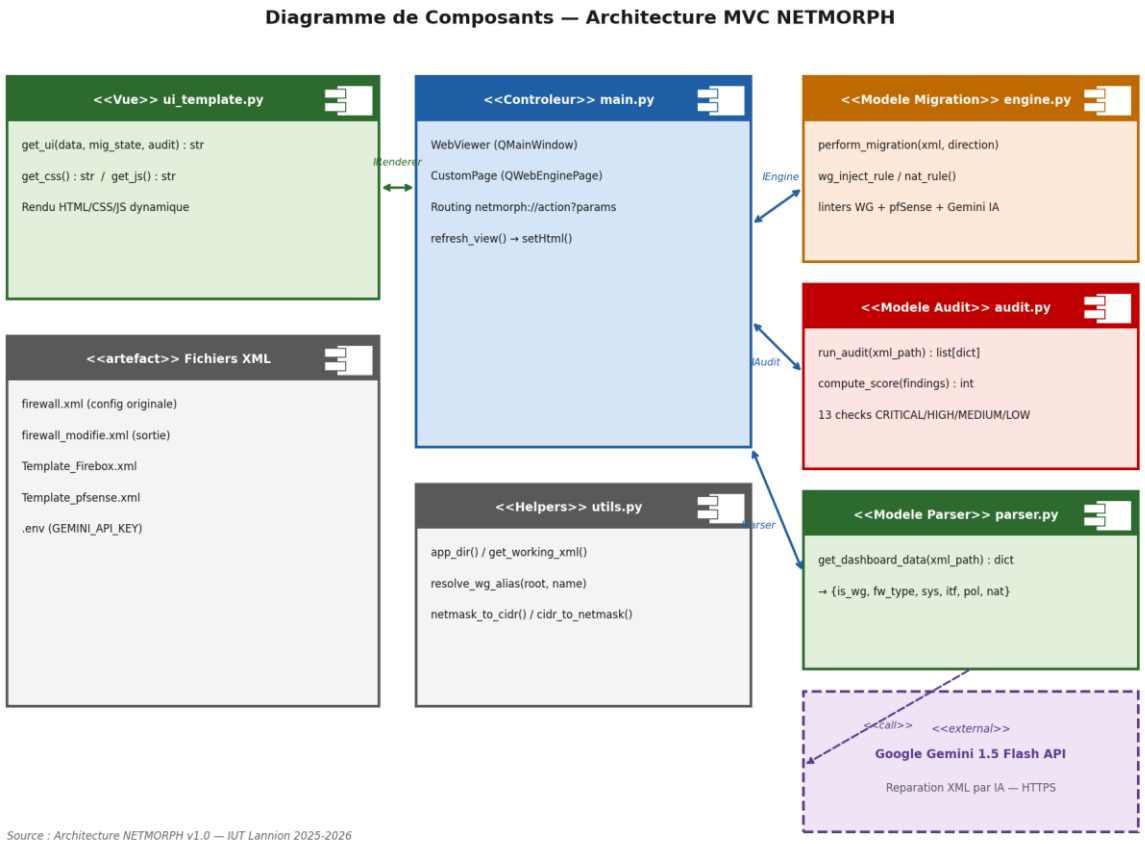


Figure 1 — Diagramme de composants : architecture MVC NETMORPH, interfaces et dépendances

La communication entre la Vue HTML (rendue dans QWebEngineView) et le code Python se fait via le protocole IPC netmorph://. Ce schéma d'URL propriétaire est intercepté par CustomPage.acceptNavigationRequest() qui parse l'action et les paramètres avant de router vers la méthode Python appropriée. Le retour False bloque la navigation effective dans le navigateur, maintenant la page en place tout en exécutant l'action demandée.

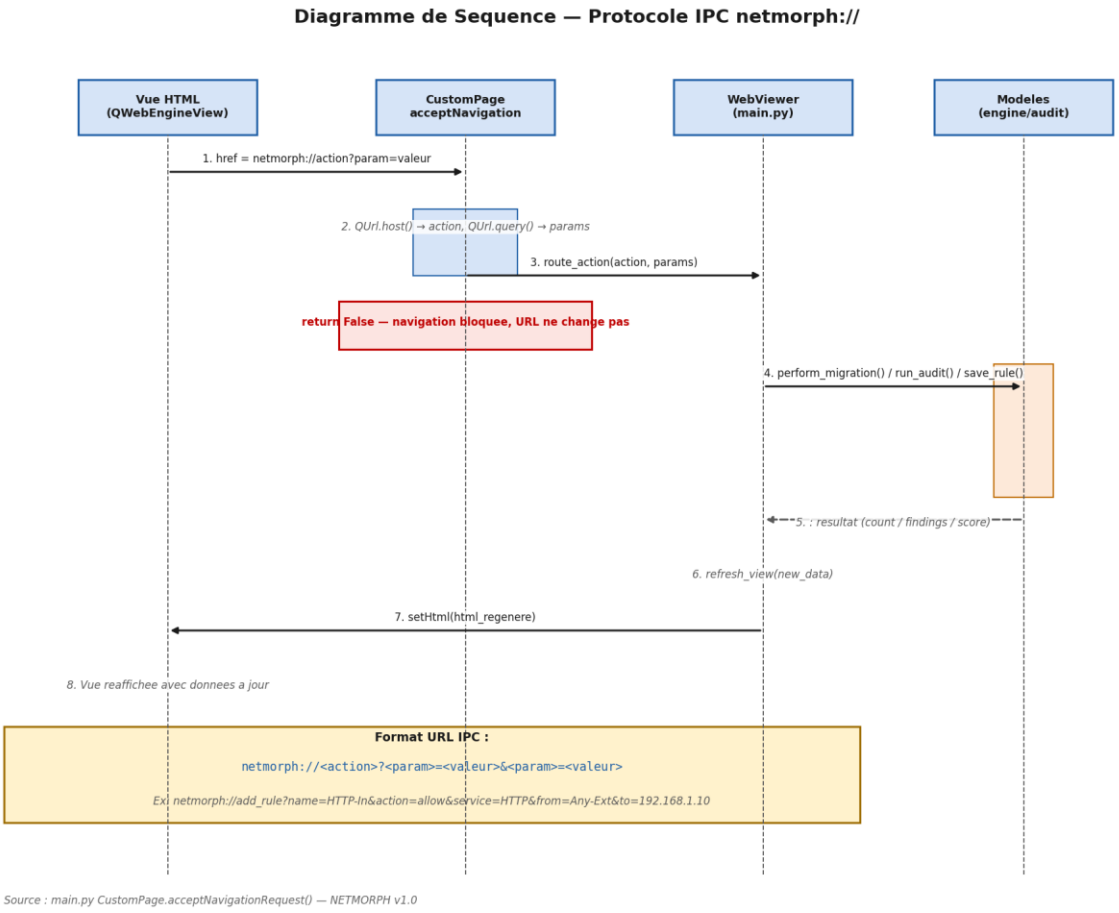


Figure 2 — Diagramme de séquence : protocole IPC netmorph:// Vue ↔ Contrôleur ↔ Modèle

2. Vue métier — Migration WatchGuard → pfSense

Ce flux décrit en détail les 11 étapes de la migration automatisée depuis une configuration WatchGuard Fireware vers pfSense. Chaque étape est localisée dans le module responsable avec ses entrées et sorties.

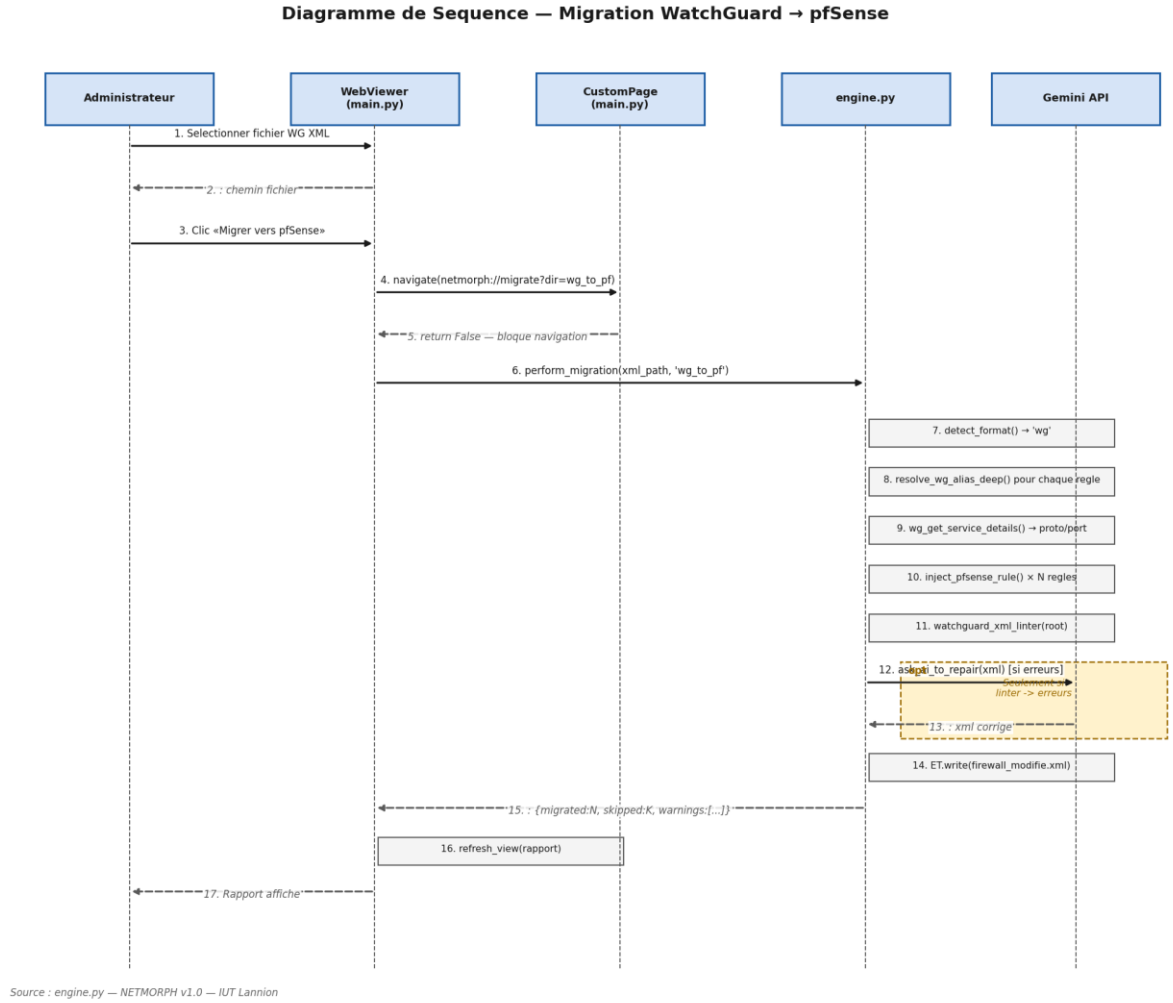


Figure 3 — Diagramme de séquence : migration WatchGuard → pfSense détaillée

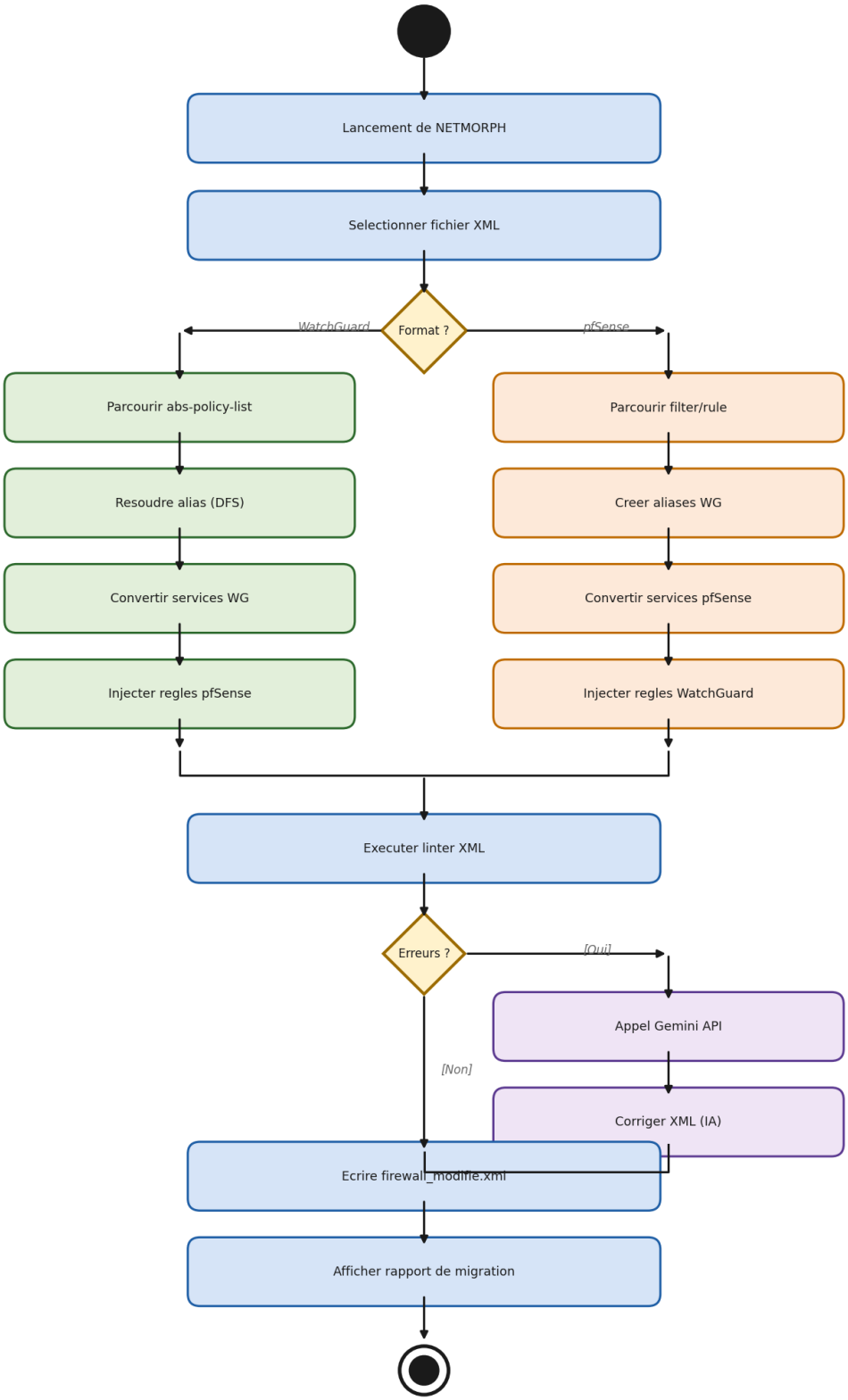
La migration débute par la détection du type source (root.tag == 'profile'), suivie du chargement en mémoire du template pfSense vierge. Pour chaque abs-policy non système (property ≠ 32), l'algorithme résout récursivement les alias source et destination en IPs réelles, mappe le service WatchGuard en protocole/port pfSense, vérifie si une règle NAT type=7 est associée (SNAT → port forwarding pfSense), puis injecte la règle filter/rule avec l'ipprotocol obligatoire. Les VLANs (interface vlan-if) et les aliases custom sont également migrés dans leurs sections respectives.

À l'issue de la migration, perform_migration() retourne un dictionnaire rapport contenant : count (règles migrées), vlans (liste des VLANs), aliases (liste des aliases créés), skipped (règles système ignorées), warnings (liste des avertissements non bloquants) et rules (noms des règles migrées). Ce rapport est affiché dans une QMessageBox.information().

3. Vue métier — Migration pfSense → WatchGuard

La migration pfSense → WatchGuard est architecturalement plus complexe car WatchGuard Fireware requiert des structures XML beaucoup plus élaborées : double nœud pour chaque règle (abs-policy + policy moteur), objets réseau wrappés (address-group + alias), et structure SNAT en 3 parties. NETMORPH crée automatiquement toutes ces structures lors de la migration.

Diagramme d'Activité — Processus de Migration NETMORPH



Source : engine.py — NETMORPH v1.0 — IUT Lannion

Figure 4 — Diagramme d'activité : processus complet de migration avec gestion des erreurs IA

Pour chaque règle filter/rule pfSense non désactivée (<disabled/> absent), l'engine extrait source, destination, protocole et port. Pour chaque IP source et destination non standard, ensure_wg_ip_alias() crée un address-group (container IP) et un alias (wrapper référençable dans les policies). Pour les ports non référencés dans la service-list native WatchGuard, ensure_wg_custom_service() crée un service custom TCP-XXXX ou UDP-XXXX avec la structure XML complète. Si une règle nat/rule pfSense est associée, wg_inject_nat_rule() construit les 3 objets SNAT WatchGuard. Enfin, wg_inject_rule() crée le double nœud abs-policy + policy en respectant toutes les conventions de nommage et de propriétés WatchGuard.

4. Vue détaillée — Résolution récursive d'alias

L'algorithme de résolution d'alias est l'un des composants les plus critiques de NETMORPH. WatchGuard permet des chaînes d'alias de profondeur arbitraire, et les configurations d'entreprise contiennent fréquemment des alias qui référencent d'autres alias qui référencent des address-groups. Sans résolution récursive, il serait impossible de retrouver les IPs réelles nécessaires à la génération des règles pfSense.

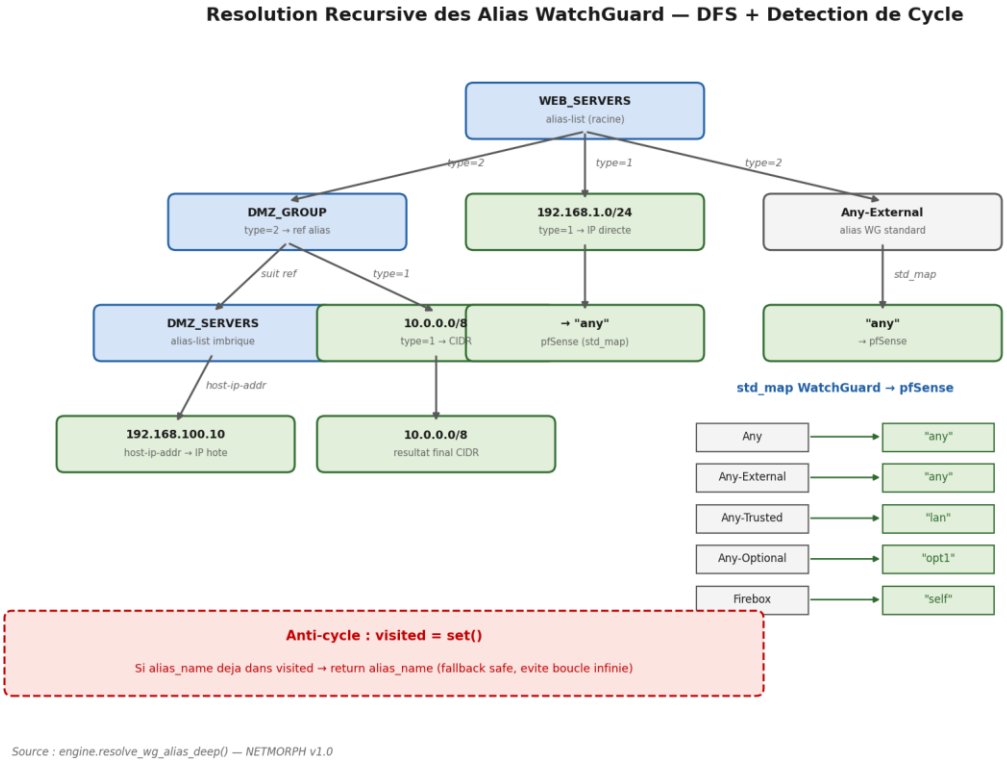


Figure 5 — Résolution récursive des alias WatchGuard : arbre DFS avec anti-cycle

L'algorithme DFS maintient un ensemble 'visited' qui accumule les noms d'alias déjà traversés. Si un alias est rencontré une seconde fois dans la même chaîne de résolution, l'algorithme retourne immédiatement le nom sans continuer — ceci prévient toute boucle infinie en cas de références circulaires dans la configuration. La profondeur typique de résolution est de 2 à 3 niveaux dans les configurations réelles. L'ensemble visited est initialisé à None par défaut et instancié uniquement au premier appel (paramètre _visited), ce qui le rend thread-safe et réentrant.

5. Vue détaillée — SNAT WatchGuard

La structure SNAT WatchGuard (Static NAT, nat type=7) est l'exemple le plus complexe de la migration pfSense → WatchGuard. Là où pfSense utilise une simple règle <nat><rule> avec target et local-port, WatchGuard nécessite la création de 3 objets XML distincts et inter-liés par convention de nommage.

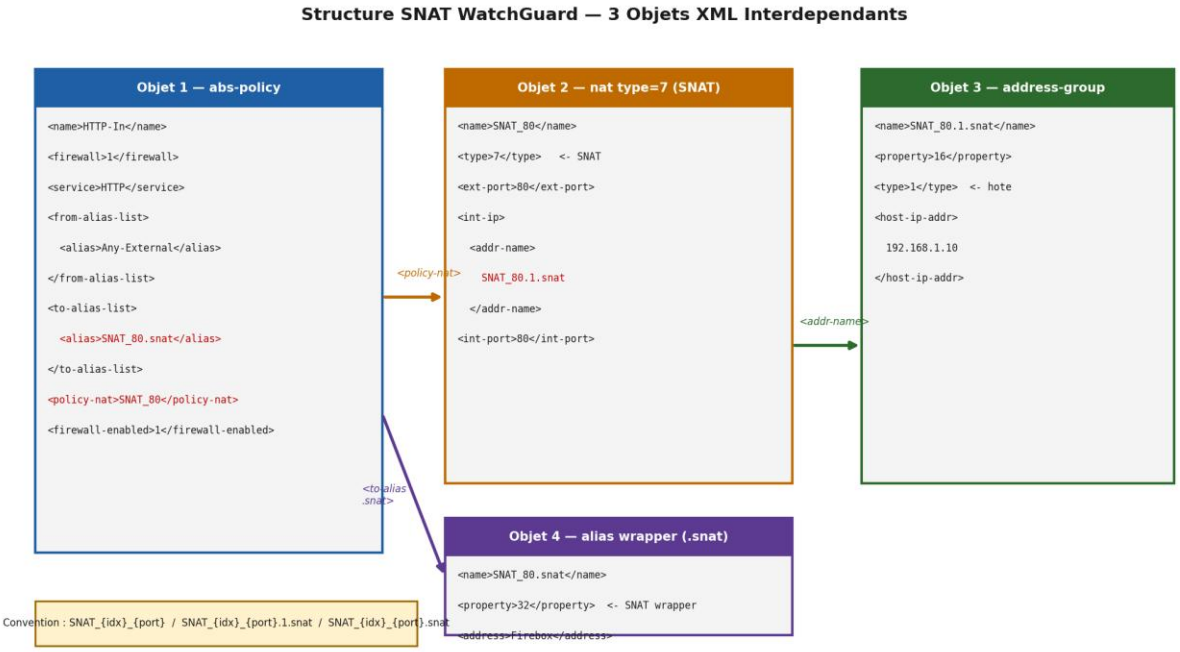


Figure 6 — Structure SNAT WatchGuard : 3 objets XML interdépendants et liaisons

L'objet 1 (address-group .1.snat) stocke l'IP interne cible. L'objet 2 (nat type=7) contient la règle de redirection : port externe WAN, référence à l'address-group (.1.snat) et interface externe. L'objet 3 (alias .snat, property=32) est un wrapper système référencé dans la <to-alias-list> de la policy correspondante. La property=32 indique à WatchGuard que cet alias est un objet système géré automatiquement — il ne doit pas être modifié manuellement dans la Web UI.

6. Vue infrastructure

NETMORPH est une application entièrement locale. Elle ne requiert aucune infrastructure serveur, aucune base de données et aucune connexion réseau pour son fonctionnement de base. Les seules communications réseau sont les appels API optionnels vers Gemini (Google AI) pour l'auto-réparation XML, qui nécessitent une configuration explicite par l'administrateur via le fichier .env.

En mode compilé (PyInstaller), la résolution de app_dir() retourne os.path.dirname(sys.executable) — le dossier de l'exécutable. En mode source, elle retourne os.path.dirname(os.path.abspath(__file__)) — le dossier du script. Cette abstraction garantit que l'application trouve ses ressources (templates, logo, .env) dans le même dossier que l'exécutable ou le script, quelle que soit la méthode d'exécution.

7. Vue sécurité

L'architecture de sécurité de NETMORPH est fondée sur le principe de défense en profondeur appliqué aux données de configuration firewall qui sont intrinsèquement sensibles : elles révèlent la topologie réseau, les serveurs internes, les politiques d'accès et les éventuelles failles de configuration de l'organisation.

La première ligne de défense est l'isolation locale complète : aucune donnée de configuration n'est transmise en dehors du poste de travail. La deuxième ligne est la validation systématique : chaque fichier importé est validé par inspection de sa balise racine, et chaque XML généré est vérifié par un linter avant persistance. La troisième ligne est la préservation de l'intégrité : le fichier original (firewall.xml) n'est jamais modifié — toutes les opérations écrivent dans un fichier séparé (firewall_modifie.xml) permettant un rollback immédiat.